ENGR 102 Summer Bridge
Day 14 Instructor Key

Lists, Length, Indexes, Mutation, and List Loops

Thursday, July 23, 2026 • 50 points

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Daily target — increasing independence

Choose index evidence when stored values must be read, changed, summarized, and located.

Allowed tools	Prior tools plus lists, len, append, indexed reads, and indexed assignment
Support supplied	A starting list and required final-state summaries are supplied.
You must decide	Loop form, list mutation, summary state, and first-largest tracking.
Scaffold level	Specification only; no planning prompts or variable inventory.

Scoring and completion standard — 50 points

Evidence	Points	Secure work shows
Integrated trace	10	intermediate state, condition results, and exact output
Diagnosis and repair	8	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	32	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace — 10 points

Trace index state, list values, an appended evidence list, and an accumulator in one ledger.

```
values = [6, 3, 9, 4]
total = 0
low_positions = []

for index in range(len(values)):
    if values[index] < 5:
        low_positions.append(index)
    else:
        total = total + values[index]

print(total, low_positions, values[len(values) - 1])
```

Your task

1. Give the complete index-by-index ledger and exact output, distinguishing each index from its stored value.

Answer and explanation

1. Index 0 stores 6, so total becomes 6 and low_positions stays empty. Index 1 stores 3, so index 1 is appended. Index 2 stores 9, so total becomes 15. Index 3 stores 4, so index 3 is appended. The last valid index is len(values) - 1 = 3, which stores 4. Exact output: 15 [1, 3] 4.

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair — 8 points

The intent is to count every negative value, including the first and last positions.

```
values = [-2, 4, -1, 6, -3]
negative_count = 0

for index in range(1, len(values) - 1):
    if values[index] < 0:
        negative_count = negative_count + 1

print(negative_count)
```

Your task

1. Give actual versus intended result, identify both skipped boundaries, repair the loop, and name one compact test that protects both ends.

Answer and explanation

1. The loop visits indexes 1, 2, and 3, so it counts only -1 and prints 1. The intended count is 3 because indexes 0, 2, and 4 are negative. Repair with range(len(values)). Protective test: [-1, 2, -3] must produce 2; it fails if either first or last index is skipped.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program — 32 points across Pages 4-5

Program specification

- Use readings = [12, -3, 7, 20, 7]. The list is nonempty.
- Loop by index and replace every negative value with 0.
- Using final values, compute valid_total and the number of values at or above 10.
- Find the index of the first largest final value without using max or index.
- Print the final list, valid_total, high_count, and largest_index on separate lines.

Answer and explanation

```
readings = [12, -3, 7, 20, 7]
valid_total = 0
high_count = 0
largest = readings[0]
largest_index = 0

for index in range(len(readings)):
    if readings[index] < 0:
        readings[index] = 0

    value = readings[index]
    valid_total = valid_total + value
    if value >= 10:
        high_count = high_count + 1
    if value > largest:
        largest = value
        largest_index = index

print(readings)
print(valid_total)
print(high_count)
print(largest_index)
```

Mutation occurs before the summaries, so total, threshold count, and largest state all describe the final list.

The strict greater-than comparison preserves the first largest index if a tie occurs. An index loop is required because the program both writes to positions and reports a position.

Task 3 continued. Test and defend the program **included in 32 points**

Expected tests and results

1. The supplied list becomes [12, 0, 7, 20, 7] and prints total 46, high_count 2, and largest_index 3.
2. Tie test [9, -1, 9] becomes [9, 0, 9] and must keep largest_index 0.
3. Boundary test [-2, 4, -1] becomes [0, 4, 0] and protects first/last mutation.

Scoring guidance

5 initialization; 5 complete indexed traversal; 5 negative mutation; 5 final-state total/count; 6 first-largest tracking; 3 output; 3 tests/explanation.

Accept alternative correct code when it obeys the stated tool boundary, handles the required cases, and produces the required output. All blue text is answer or scoring guidance.

VIP Lab Alignment

Bridge Day 14 • Contract 7cc3f8d502f3

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

Why this page is here

VIP Lab Day(s): 6 **Activities:** 18.25, 18.26, 18.27

Carry index, value, and final-list evidence into reverse traversal and one- or two-condition filters.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.25 18-25-work / 18-25-transfer / 18-25-cold	**Reverse Values**	Create a new list without mutating the input.
18.26 18-26-work / 18-26-transfer / 18-26-cold	**Values At Or Above Cut-off**	Filter without reordering.
18.27 18-27-work / 18-27-transfer / 18-27-cold	**Feasible Workstations**	Read supplied workstation records.

Readiness check before you code

1. State which mapped activity requires indexes and which activities can preserve order by traversing values.

   Answer and explanation: 18.25 requires reverse index traversal. Activities 18.26 and 18.27 can traverse records or values in original order and append only passing results.

2. For Activity 18.27, write the two Boolean constraints separately before combining them with `and`.

   Answer and explanation: The exact constraints come from the supplied workstation records and task. Both comparisons must be true before the workstation name is appended.

Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook cold cells without a reveal or copied solution. Only those cold cells are scored for independent mastery on Lab Days 5–7.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.